

Звіт з лабораторної роботи №8

Тема: Розширені можливості Redis

Студентка: Гульчак Дар'я, група МІТ-31

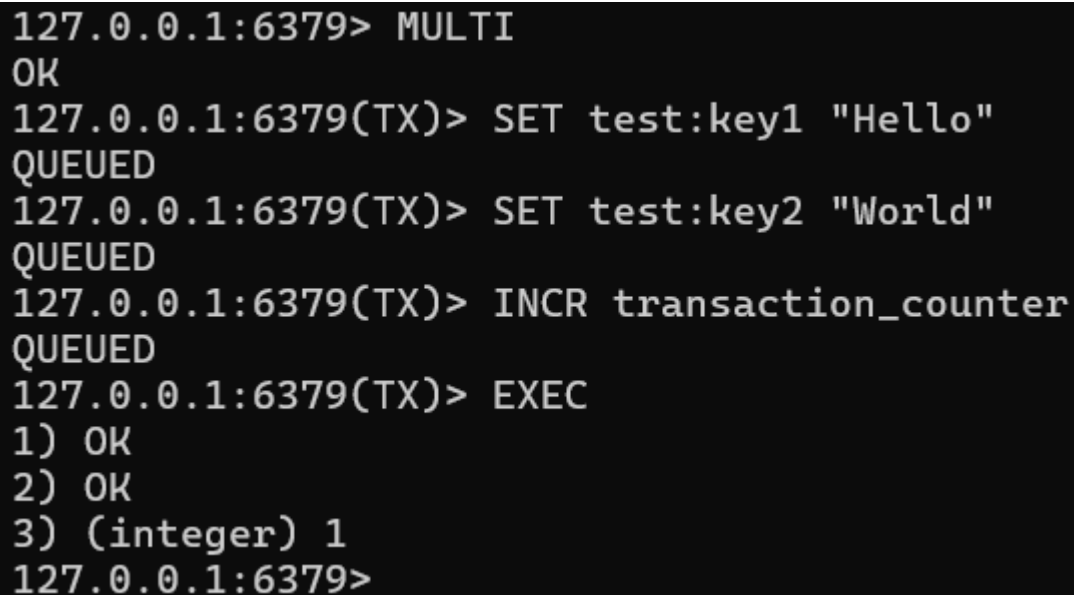
1. Вступ

Метою роботи є ознайомлення з розширеним функціоналом Redis: транзакціями, механізмом Optimistic Locking через `WATCH`, написанням скриптів мовою Lua, а також вивчення моделей Pub/Sub та Streams для обробки подій у реальному часі.

2. Практична частина

2.1. Транзакції (MULTI/EXEC/WATCH)

Було реалізовано атомарне виконання команд. Використання команди `WATCH` дозволило забезпечити цілісність даних при конкурентному доступі.

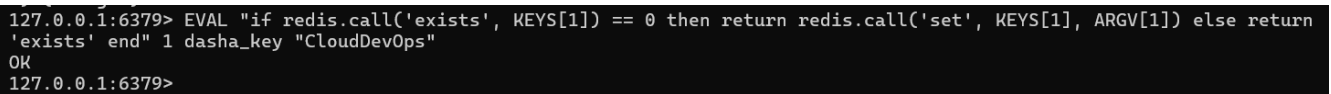


```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET test:key1 "Hello"
QUEUED
127.0.0.1:6379(TX)> SET test:key2 "World"
QUEUED
127.0.0.1:6379(TX)> INCR transaction_counter
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) OK
3) (integer) 1
127.0.0.1:6379>
```

Рисунка 1: Скріншот транзакції

2.2. Lua-скрипти

Написано та протестовано Lua-скрипт для атомарної перевірки та створення ключа. Це дозволяє зменшити мережеві затримки.



```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 dasha_key "CloudDevOps"
OK
127.0.0.1:6379>
```

Рисунка 2: Виконання EVAL

2.3. Pub/Sub та Streams

- **Pub/Sub:** Реалізовано передачу повідомлень між двома незалежними клієнтами через канал `news`.
- **Streams:** Створено потік подій `mystream` та зчитано дані за допомогою `XRANGE`.

```
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key to type command)
```

```
PS C:\Users\HP Victus> docker exec -it redis-lab redis-cli
127.0.0.1:6379> PUBLISH news "Dasha completed step 3!"
(integer) 1
127.0.0.1:6379>
```

```
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
1) "message"
2) "news"
3) "Dasha completed step 3!"
Reading messages... (press Ctrl-C to quit or any key to type command)
```

Рисунка 3: Робота Pub/Sub у двох вікнах

```
127.0.0.1:6379> XADD mystream * sensor-id 777 temperature 22.5
"1778599228716-0"
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1778599228716-0"
    2) 1) "sensor-id"
        2) "777"
        3) "temperature"
        4) "22.5"
127.0.0.1:6379>
```

Рисунка 4: Робота з Redis Streams

3. Відповіді на запитання для самоперевірки

1. **WATCH** потрібна для реалізації "оптимістичного блокування". Якщо ключ, за яким ми "спостерігаємо", зміниться іншим клієнтом до виконання транзакції, транзакція буде скасована.
2. **Lua-скрипти** гарантують атомарність (весь скрипт виконується як одна операція) і знижують навантаження на мережу.

3. **Redis Streams** відрізняються від Pub/Sub тим, що вони зберігають історію повідомлень (persistence), тоді як у Pub/Sub повідомлення зникає, якщо підписник був офлайн.

4. Висновок

Розширені можливості Redis роблять його потужним інструментом для DevOps. Транзакції та Lua забезпечують надійність даних, а Streams дозволяють будувати складні системи обробки подій, що є альтернативою важким брокерам повідомлень.